

22/01/2026 Learning Go to make another side project using prabogo an the whole merah putih framework because good god it would be so fucking funny!!!!!! Uwu

Advantages of Go :

Go	Python	C++
Statically typed	Dynamically typed	Statically typed
Fast run time	Slow run time	Fast run time
Compiled	Interpreted	Compiled
Fast compile time	Interpreted	Slow compile time
Supports concurrency through goroutines and channel	No built-in concurrency mechanism	Supports concurrency through threads
Has automatic garbage collection	Has automatic garbage collection	Does not have automatic garbage collection
Does not support classes and objects	Has classes and objects	Has classes and objects
Does not support inheritance	Supports inheritance	Supports inheritance

It also have concurrency unlike JS so you don't have to use longass callback functions for the backend.

```
//hello_world.go
package main
import ("fmt")

func main() {
    fmt.Println("Hello World!")
}
```

Line 1: In Go, every program is part of a package. We define this using the package keyword. In this example, the program belongs to the main package.

Line 2: import ("fmt") lets us import files included in the fmt package.

Line 3: func main() {} is a function. Any code inside its curly brackets {} will be executed.

Line 5: fmt.Println() is a function made available from the fmt package. It is used to output/print text. In our example it will output "Hello World!".

Variables :

2 ways to declare :

Var :

```
var variablename type = value
```

:= sign -> inferred values from the values

```
variablename := value
```

Eg:

```
// Variable declaration and initialization
```

```
var (
    a int = 6
    b string = "Hello"
)
c, d := 7, "World!"
```

Constants :

Similar to js const but it allows static typing

```
package main
import ("fmt")

const (
    A int = 1
    B = 3.14
    C = "Hi!"
)

func main() {
    fmt.Println(A)
    fmt.Println(B)
    fmt.Println(C)
}
```

Print functions :

Go has three functions to output text:

- Print() -> print without line
- Println() -> print with line
- Printf() -> print with format

REGEX formats

General format		
Verb	Description	
%v	Prints the value in the default format	
%#v	Prints the value in Go-syntax format	
%T	Prints the type of the value	
%%	Prints the % sign	
Integer formats :		Str formats :

Verb	Description	Verb	Description
%b	Base 2	%s	Prints the value as plain string
%d	Base 10	%q	Prints the value as a double-quoted string
%+d	Base 10 and always show sign	%8s	Prints the value as plain string (width 8, right justified)
%o	Base 8	%-8s	Prints the value as plain string (width 8, left justified)
%O	Base 8, with leading 0o	%x	Prints the value as hex dump of byte values
%x	Base 16, lowercase	% x	Prints the value as hex dump with spaces
%X	Base 16, uppercase		
%#x	Base 16, with leading 0x		
%4d	Pad with spaces (width 4, right justified)		
%-4d	Pad with spaces (width 4, left justified)		
%04d	Pad with zeroes (width 4)		

Bool formats		Float formats	
%t	Value of the boolean operator in true or false format (same as using %v)	Verb	Description
The default value of a boolean data type is false.		%e	Scientific notation with 'e' as exponent
		%f	Decimal point, no exponent
		%.2f	Default width, precision 2
		%6.2f	Width 6, precision 2
		%g	Exponent as needed, only necessary digits

ARRAYS

Unlike python, Go Arrays cannot be of different types

2 ways to declare : using var or :=

Using var keyword

```
var array_name = [length]datatype{values} // here length is defined
```

or

```
var array_name = [...]datatype{values} // here length is inferred
```

Using the := sign:

```
array_name := [length]datatype{values} // here length is defined
```

or

```
array_name := [...]datatype{values} // here length is inferred
```

Eg :

```
var arr1 = [4]int{1,2,3}
arr2 := [...]int{4,5,6,7,8}
arr3 := [5]int{1:10,2:40}
```

```
fmt.Println(arr1)
fmt.Println(arr2)
fmt.Println(arr3)
fmt.Println(len(arr3))
}
```

Slices

Slices are similar to arrays, but are more powerful and flexible.

Like arrays, slices are also used to store multiple values of the same type in a single variable.

However, unlike arrays, the length of a slice can grow and shrink as you see fit.

In Go, there are several ways to create a slice:

- Using the `[]datatype{values}` format
- Create a slice from an array
- Using the `make()` function

```
myslice1 := []int{}
fmt.Println(len(myslice1))
fmt.Println(cap(myslice1))
fmt.Println(myslice1)

myslice2 := []string{"Go", "Slices", "Are", "Powerful"}
fmt.Println(len(myslice2))
fmt.Println(cap(myslice2))
fmt.Println(myslice2)
```

- `len()` function - returns the length of the slice (the number of elements in the slice)
- `cap()` function - returns the capacity of the slice (the number of elements the slice can grow or shrink to)

Slicing from an array (see where did it get it's name):

```
arr4 := [6]int{10, 11, 12, 13, 14, 15}
myslice := arr4[2:4]
```

Using make function :

```
slice_name := make([]type, length, capacity)
```

Operators

Arithmetic	Logical	Bitwise Logical
------------	---------	-----------------

Operator	Name	Operator	Name	Operator	Name
+	Addition	&&	Logical and	&	AND
-	Subtraction		Logical or		OR
*	Multiplication	!	Logical not	^	XOR
/	Division			<<	Zero fill left shift
%	Modulus			>>	Signed right shift
++	Increment				
--	Decrement				

Conditional

```

if condition1 {
    // code to be executed if condition1 is true
} else if condition2 {
    // code to be executed if condition1 is false and condition2 is true
} else {
    // code to be executed if condition1 and condition2 are both false
}

```

Loops

There's only for loops in Go

```

// for loop
for i := 0; i < 9; i++ {
    if i == 3 {
        continue
    }
    fmt.Println(i)
    if i == 6 {
        break
    }
}
fmt.Println("")
// enumerate over slice
fruits := []string{"apple", "orange", "banana"}
for idx, val := range fruits {
    fmt.Printf("%v\t%v\n", idx, val)
}

```

Functions

HASHMAP !!!!

2 implementations also using make and direct

Direct implementations :

```
package main
import ("fmt")

func main() {
    var a = map[string]string{"brand": "Ford", "model": "Mustang", "year": "1964"}
    b := map[string]int{"Oslo": 1, "Bergen": 2, "Trondheim": 3, "Stavanger": 4}

    fmt.Printf("a\t%\v\n", a)
    fmt.Printf("b\t%\v\n", b)
}
```

Make Implementations :

```
// using maps
var x = map[string]string{"brand": "Ford", "model": "Mustang", "year": "1964"}
// using make to create map
y := make(map[string]int)
y["Oslo"] = 1
y["Bergen"] = 2
y["Trondheim"] = 3
y["Stavanger"] = 4

delete(x, "year")
// add elements to map
x["color"] = "Red" // Adding an element

// access elements from map
_, ok4 := x["model"] // Only checking for existing key and not its value
fmt.Println(ok4)
// delete element from map

fmt.Printf("x\t%\v\n", x)
fmt.Printf("y\t%\v\n", y)

// took as reference
z := x
z["color"] = "Blue"
fmt.Printf("x\t%\v\n", x)
fmt.Printf("z\t%\v\n", z)

// iterate over map
for k, v := range x {
    fmt.Printf("%v : %v, ", k, v)
}
}
```